

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG
PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI



TRẦN PHƯỚC TÀI
PHAN TRẦN TƯỜNG VY

DỰ ĐOÁN KHÁCH HÀNG VÀ KHU VỰC TIỀM NĂNG

ĐỒ ÁN HỌC PHẦN HỌC MÁY

TP. HỒ CHÍ MINH, NĂM 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG
PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI

TRẦN PHƯỚC TÀI
PHAN TRẦN TƯỜNG VY

DỰ ĐOÁN KHÁCH HÀNG VÀ KHU VỰC TIỀM NĂNG

Ngành : Công Nghệ Thông Tin

Mã số:

NGƯỜI HƯỚNG DẪN 1. ThS. Vũ Thi Hạnh

TP. HỒ CHÍ MINH, NĂM 2026

LỜI CAM ĐOAN

Tôi xin cam đoan đây là Đồ án học phần của bản thân tôi. Các kết quả trong Đồ án học phần này là trung thực, và không sao chép từ bất kỳ một nguồn nào và dưới bất kỳ hình thức nào. Việc tham khảo các nguồn tài liệu (nếu có) đã được thực hiện trích dẫn và ghi nguồn tài liệu tham khảo đúng quy định.

MỤC LỤC

DANH MỤC CÁC HÌNH ẢNH.....	iii
DANH MỤC BẢNG BIỂU.....	iv
DANH MỤC CÁC TỪ VIẾT TẮT VÀ GIẢI THÍCH CÁC THUẬT NGỮ	v
CHƯƠNG 1 GIỚI THIỆU ĐỀ TÀI	1
CHƯƠNG 2 MÔ TẢ DỮ LIỆU VÀ TIỀN XỬ LÝ DỮ LIỆU	2
2.1 Mô tả dữ liệu.....	2
2.2 Tiền xử lý dữ liệu	3
CHƯƠNG 3 HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH.....	4
3.1 Huấn luyện.....	4
3.1.1 K_Means.....	4
3.1.2 Logistic Regression & Random Forest	5
3.1.3 Gradient Bootsting.....	6
3.2 Đánh giá mô hình	8
PHỤ LỤC	9

DANH MỤC CÁC HÌNH ẢNH

No table of figures entries found.

DANH MỤC BẢNG BIỂU

DANH MỤC CÁC TỪ VIẾT TẮT VÀ GIẢI THÍCH CÁC THUẬT NGỮ

CHƯƠNG 1 GIỚI THIỆU ĐỀ TÀI

Đồ án này tập trung nghiên cứu tầng giao vận (Transport Layer) trong mô hình TCP/IP, cụ thể là hai giao thức chính: TCP (Transmission Control Protocol) và UDP (User Datagram Protocol). TCP là giao thức hướng kết nối, đảm bảo truyền dữ liệu đáng tin cậy nhờ cơ chế thiết lập kết nối ba bước (three-way handshake), kiểm soát lỗi, kiểm soát luồng dữ liệu và sắp xếp thứ tự gói tin, phù hợp với các ứng dụng yêu cầu độ chính xác cao như HTTP, FTP và email. Ngược lại, UDP là giao thức không kết nối, nhanh chóng và nhẹ hơn, không đảm bảo độ tin cậy nhưng ưu tiên tốc độ truyền, thường được sử dụng trong DNS, streaming video hoặc các ứng dụng thời gian thực. Ngoài ra, nghiên cứu đề cập đến kỹ thuật ghép kênh (multiplexing) thông qua số hiệu cổng (port), với các cổng phổ biến như: HTTP (80/TCP), FTP (20-21/TCP), DNS (53/UDP hoặc TCP), và các dịch vụ email (SMTP: 25/TCP, POP3: 110/TCP, IMAP: 143/TCP). Để minh họa, mô hình mạng được thiết kế với một Multi-Server kết nối qua Switch đến các client: HTTP Client, FTP Client, E-Mail Client và DNS Client, nhằm tạo lưu lượng mạng trong chế độ mô phỏng và phân tích thực tế chức năng của TCP và UDP trong môi trường mạng LAN.

CHƯƠNG 2 MÔ TẢ DỮ LIỆU VÀ TIỀN XỬ LÝ DỮ LIỆU

2.1 Mô tả dữ liệu

```
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer ID            3900 non-null   int64
1   Age                    3900 non-null   int64
2   Gender                  3900 non-null   object
3   Item Purchased          3900 non-null   object
4   Category                3900 non-null   object
5   Purchase Amount (USD)   3900 non-null   int64
6   Location                 3900 non-null   object
7   Size                    3900 non-null   object
8   Color                   3900 non-null   object
9   Season                  3900 non-null   object
10  Review Rating           3900 non-null   float64
11  Subscription Status      3900 non-null   object
12  Shipping Type            3900 non-null   object
13  Discount Applied         3900 non-null   object
14  Promo Code Used          3900 non-null   object
15  Previous Purchases       3900 non-null   int64
16  Payment Method           3900 non-null   object
17  Frequency of Purchases   3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
None
Kích thước dữ liệu: (3900, 18)
Số lượng Customer ID unique: 3900
```

2.2 Tiền xử lý dữ liệu

```
# Kiểm tra missing values
print(df.isnull().sum())

# Encode categorical columns
categorical_cols = df.select_dtypes(include=['object']).columns
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col + '_encoded'] = le.fit_transform(df[col])
    label_encoders[col] = le

# Numerical columns for scaling
numerical_cols = ['Age', 'Purchase Amount (USD)', 'Review Rating',
                  'Previous Purchases']

# Standardize numerical features
scaler = StandardScaler()
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

# Drop không liên quan: Customer ID (unique, not useful for analysis)
df = df.drop(['Customer ID'], axis=1)

print(df.head())
```

```
Customer ID      0
Age              0
Gender           0
Item Purchased   0
Category         0
Purchase Amount (USD)  0
Location         0
Size            0
Color           0
Season          0
Review Rating    0
Subscription Status  0
Shipping Type    0
Discount Applied  0
Promo Code Used  0
Previous Purchases  0
Payment Method   0
Frequency of Purchases  0
dtype: int64
```

CHƯƠNG 3 HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH

3.1 Huấn luyện

3.1.1 K_Means

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler, LabelEncoder

print("🔄 Đang tính toán lại K-Means để lấy labels...")

# 1. CHUẨN BỊ DỮ LIỆU LẠI
# Chọn các features dùng để phân cụm
features_cluster = ['Age', 'Purchase Amount (USD)', 'Review Rating',
                    'Previous Purchases']
# Nếu bạn dùng features khác, hãy sửa list trên

df_clus = df.copy()
# Mã hóa các cột phân loại nếu cần (ví dụ Gender, Subscription) để đưa
vào clustering
# Ở đây thêm Gender và Subscription vào để phân cụm rõ hơn
le = LabelEncoder()
df_clus['Gender_Code'] = le.fit_transform(df_clus['Gender'])
df_clus['Sub_Code'] = le.fit_transform(df_clus['Subscription Status'])

# Tạo X_cluster
X_cluster = df_clus[features_cluster + ['Gender_Code',
                                         'Sub_Code']].values
scaler = StandardScaler()
X_cluster_scaled = scaler.fit_transform(X_cluster)

# 2. CHẠY K-MEANS (k=2 như đã chốt)
kmeans = KMeans(n_clusters=2, random_state=42)
labels = kmeans.fit_predict(X_cluster_scaled)

print("✅ Đã có labels. Bắt đầu phân tích...")
print("-" * 50)

# 3. GẮN NHÃN VÀO DATAFRAME GỐC
df_analysis = df.copy()
df_analysis['Cluster'] = labels

# 4. PHÂN TÍCH BIẾN SỐ (Mean Values)
numeric_cols = ['Age', 'Purchase Amount (USD)', 'Review Rating',
                'Previous Purchases']
```

```

numeric_profile =
df_analysis.groupby('Cluster')[numeric_cols].mean().round(2)

print("\n📊 1. ĐẶC ĐIỂM VỀ CHỈ SỐ TRUNG BÌNH (Mean):")
display(numeric_profile)

# 5. PHÂN TÍCH BIẾN PHÂN LOẠI (Tỷ lệ %)
categorical_cols = ['Gender', 'Subscription Status', 'Frequency of
Purchases']

for col in categorical_cols:
    print(f"\n--- Phân bố {col} trong từng cụm (%) ---")
    cross_tab = pd.crosstab(df_analysis['Cluster'], df_analysis[col],
normalize='index') * 100
    display(cross_tab.round(1))

    # Vẽ biểu đồ
    cross_tab.plot(kind='barh', stacked=True, colormap='viridis',
figsize=(8, 3))
    plt.title(f'Tỷ lệ {col} theo từng Cụm')
    plt.xlabel('Phần trăm (%)')
    plt.ylabel('Cụm (Cluster)')
    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.show()
Logistic Regression
Logistic Regression
Logistic Regression

```

3.1.2 Logistic Regression & Random Forest

```

# Chuẩn bị data Churn
df['Churn'] = (df['Subscription Status'] == 'No').astype(int) # 1:
churn, 0: not

features_clf = [col for col in df.columns if col not in ['Churn',
'Subscription Status', 'Subscription Status_encoded', 'Cluster'] and
'encoded' in col or col in numerical_cols]
X_clf = df[features_clf]
y_clf = df['Churn']

X_train_clf, X_test_clf, y_train_clf, y_test_clf =
train_test_split(X_clf, y_clf, test_size=0.2, random_state=42)

# Model 1: Logistic Regression
logreg = LogisticRegression(random_state=42)
logreg.fit(X_train_clf, y_train_clf)
y_pred_log = logreg.predict(X_test_clf)

```

```

print("Logistic Regression Accuracy:", accuracy_score(y_test_clf,
y_pred_log))
print(classification_report(y_test_clf, y_pred_log))

# Model 2: Random Forest Classifier
rf_clf = RandomForestClassifier(random_state=42)
rf_clf.fit(X_train_clf, y_train_clf)
y_pred_rf_clf = rf_clf.predict(X_test_clf)
print("Random Forest Accuracy:", accuracy_score(y_test_clf,
y_pred_rf_clf))
print(classification_report(y_test_clf, y_pred_rf_clf))

```

3.1.3 Gradient Bootsting

```

# --- TASK DỰ ĐOÁN CHI TIÊU (IMPROVED VERSION) ---
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler

print("🔪 Đang phân tích bài toán Dự đoán Chi tiêu...")

# 1. FEATURE SELECTION & ENGINEERING
# Chọn Features đầu vào cụ thể (tránh lấy nhầm cột rác)
features_spending = [
    'Age', 'Gender', 'Category', 'Season', 'Subscription Status',
    'Review Rating', 'Previous Purchases'
]
target_col = 'Purchase Amount (USD)'

# Tạo bản sao
df_reg = df[features_spending + [target_col]].copy()

# One-Hot Encoding (Tốt hơn Label Encoding cho Regression)
# Giúp máy hiểu 'Category_Clothing' là 1 đặc điểm riêng biệt
df_reg = pd.get_dummies(df_reg, columns=['Gender', 'Category',
'Season', 'Subscription Status'], drop_first=True)

# 2. KIỂM TRA TƯƠNG QUAN (DIAGNOSIS)
# Bước này để trả lời tại sao R2 thấp
correlation = df_reg.corr()[target_col].sort_values(ascending=False)
print("\n📊 Mức độ tương quan với số tiền chi tiêu (Purchase Amount):")
print(correlation.head(10)) # In ra những yếu tố ảnh hưởng nhất

```

```

# Vẽ Heatmap
plt.figure(figsize=(10, 8))
# Lấy Top 10 features quan trọng nhất để vẽ cho đỡ rối
top_features = correlation.abs().nlargest(10).index
sns.heatmap(df_reg[top_features].corr(), annot=True, cmap='coolwarm',
fmt=".2f")
plt.title("Biểu đồ Tương quan (Heatmap)")
plt.show()

# 3. CHUẨN BỊ DATA TRAIN
X = df_reg.drop(target_col, axis=1)
y = df_reg[target_col]

# Chia tập train/test
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Chuẩn hóa dữ liệu (Gradient Boosting chạy tốt hơn khi data cùng
scale)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. HUẤN LUYỆN MODEL (Gradient Boosting)
# GBR thường mạnh hơn RF trong việc tìm các mối liên hệ tuyến tính nhỏ
gbr = GradientBoostingRegressor(n_estimators=200, learning_rate=0.05,
max_depth=3, random_state=42)
gbr.fit(X_train_scaled, y_train)

# 5. ĐÁNH GIÁ
y_pred = gbr.predict(X_test_scaled)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\n" + "="*40)
print(f"🏆 KẾT QUẢ MÔ HÌNH (GRADIENT BOOSTING):")
print(f"• MSE (Sai số bình phương trung bình): {mse:.2f}")
print(f"• R2 Score (Độ phù hợp): {r2:.4f}")
print("="*40)

# 6. TRỰC QUAN HÓA: DỰ ĐOÁN VS THỰC TẾ
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.5, color='purple')
# Vẽ đường chéo đồ (Dự đoán hoàn hảo)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
plt.xlabel('Thực tế (Actual Amount)')
plt.ylabel('Dự đoán (Predicted Amount)')

```

```
plt.title('Biểu đồ phân tán: Giá trị Thực tế vs Dự đoán')  
plt.show()
```

3.2 Đánh giá mô hình

PHỤ LỤC